

Trabajo Integrador

Aplicaciones de Sistemas Operativos en Tiempo Real

Universidad Tecnológica Nacional - Facultad Regional Avellaneda

Laboratorio de Sistemas Embebidos

Departamento de Ingeniería Electrónica

Albornoz Thiago, Poggi Lorenzo, Sarniguette Valentino

thiagoagustinalbornoz@impatrq.com

janinlorenzopoggi@impatrq.com

valentinosarniguette@impatrq.com

Abstract: Este informe describe el desarrollo de un sistema embebido basado en FreeRTOS utilizando un microcontrolador LPC845.

I. INTRODUCCIÓN

La siguiente introducción corresponde al trabajo integrador realizado en el marco de la materia “Aplicaciones de Sistemas Operativos en Tiempo Real”, dictada por el profesor Fabrizio Carlassara en la Universidad Tecnológica Nacional – Facultad Regional Avellaneda. En este proyecto, se abordó el desarrollo de una aplicación basada en FreeRTOS para el microcontrolador LPC845 BREAKOUT, con el objetivo de poner en práctica conceptos fundamentales de programación multitarea, control de periféricos y sincronización de recursos.

II. DESARROLLO DE CONTENIDOS

El proyecto se organizó con el objetivo de mejorar la claridad y la organización del código en un entorno de tiempo real basado en FreeRTOS. A continuación, se detallan las distintas etapas y componentes que conforman su desarrollo.

En primer lugar, se trabajó sobre el archivo **labels.h**, donde se definieron etiquetas de manera estructurada y significativa. Esto favoreció una comprensión más clara del código, facilitando tanto su interpretación como su mantenimiento.

Posteriormente, se implementaron los archivos **wrappers.c** y **wrappers.h**, cuyo propósito consiste en inicializar y encapsular las APIs proporcionadas por el SDK. A través de estos wrappers se logra abstraer y simplificar el acceso a las funciones del sistema, centralizando su utilización y promoviendo una codificación más ordenada y eficiente.

En cuanto a la administración del sistema multitarea, se desarrollaron los archivos **app_task.c** y **app_task.h**. En ellos se definieron las prioridades de las tareas, los tamaños de sus pilas y los prototipos de las funciones asociadas a cada una. Además, se especificó el propósito funcional de cada tarea dentro del sistema y se realizó la inicialización de **semáforos** y **colas**, recursos indispensables para la sincronización y la comunicación entre tareas. Esta organización permitió estructurar un sistema multitarea robusto y escalable, adecuado para entornos de tiempo real.

Finalmente, en el archivo **main.c** se llevó a cabo la configuración inicial del sistema. Se estableció el reloj a **30 MHz** y se procedió a inicializar la consola de depuración. Seguidamente, se crearon múltiples tareas mediante la función **xTaskCreate()**, asignando a

cada una su función correspondiente, nombre identificatorio, tamaño de pila, prioridad y puntero de control. Estas tareas fueron diseñadas para gestionar distintos módulos funcionales del sistema, tales como sensores, display, consola, control y salidas. Por último, se ejecutó la función **vTaskStartScheduler()**, que permite iniciar el planificador de FreeRTOS y dar comienzo a la ejecución concurrente de las tareas previamente definidas.

Esta estructura no solo facilita la comprensión del código, sino que también permite una mayor flexibilidad a la hora de realizar expansiones o modificaciones en el sistema.

III. TAREAS

tarea_setpoint: ajusta el setpoint de luminosidad con S1 y S2 (25% a 75%)

tarea_adc: activa una secuencia de conversión cada 0.25 segundos

tarea_display_change: lee los valores de los botones para definir qué valor mostrar

tarea_control: escribe los valores en el display

tarea_display: escribe un número en el display

tarea_pwm: actualiza el duty del pwm

tarea_bh1750: lee periódicamente el valor de intensidad lumínica

tarea_buzzer: hace sonar el buzzer

tarea_counter_btms: controla manualmente el contador

tarea_cny70: hace sonar el buzzer cuando el sensor infrarrojo detecta movimiento

tarea_leds_tricolor: hace que los LEDs tricolores se comporten como un led rojo si la luminosidad es mayor al setpoint, azul si es menor.

TABLA I

HARDWARE		
Unidad	Componente	Esquemático
1	Led Azul	D1
1	Led tricolor	Q2, Q3, Q4 + R13-R21
1	Sensor infrarrojo	D2 (CNY70)
1	BH1750	GY-BH1750 (CN8)
1	Buzzer	B1
1	Display 7 Segmentos	DS1
2	Potenciometro	RV21
3	Botones	USER, S1 y S2

Componentes electrónicos empleados en el circuito con el **LPC845-BRK**, entre ellos sensores, LEDs, botones, buzzer y display, que permiten la interacción, visualización y monitoreo dentro del sistema.

IV. CONCLUSIONES

La realización y desarrollo de este trabajo integrador nos permitió consolidar y trabajar profundamente los conocimientos adquiridos en el curso de Sistemas Embebidos, particularmente en la **utilización de FreeRTOS** para la **administración de tareas** y el funcionamiento de **semáforos** y **colas**. Hemos logrado un **mejor entendimiento y aprendizaje del lenguaje C** en el contexto de microcontroladores y sistemas en tiempo real.

Este proyecto no sólo afianzó nuestras habilidades técnicas, sino que también reforzó significativamente nuestra capacidad de **trabajo en equipo** y **organizar tareas**, todo ello con el propósito de lograr un sistema funcional. Se logró diseñar e implementar una aplicación embebida con múltiples tareas, resolviendo diferentes problemas y desafíos entre sensores, indicadores y otros periféricos.

V. REFERENCIAS

- [1] Actividades desarrolladas durante el curso, tanto en clases teóricas como en prácticas de laboratorio.
- [2] Repositorio dedicado al microcontrolador con ejemplos para el curso: github.com/utn-fra-lse/lpc845.
- [3] Video explicativo externo sobre conceptos de FreeRTOS: youtube.com/watch?v=s5DnmeOwkxo

